# Memoria Práctica 3 ADSOF

Álvaro Gallego Serbán y Ana Olsson Rodríguez, Grupo 2263

# Índice

Índice....1
1. Decisiones en el desarrollo ....2
1.1. Decisiones de la clase Mensaje....3
1.2. Decisiones en la clase Enlace ....3
1.3. Decisiones sobre la clase Red Social ....4
1.4. Decisiones en la clase Usuario....4
1.5 Decisiones clase Usuario Interesado....5
1.6 Decisiones clase Enlace Señuelo ....5
1.7 Decisiones clase Mensaje Controlado ....5
2. Problemas afrontados....6
2.1. Atributos adicionales en la clase Mensaje ....7
2.2. Impresión por pantalla al difundir mensajes ....7
2.3. Lectura y escritura de Red Social ....8
3. Problemas pendientes....8
4. Diagrama de clases ....9
5. Pruebas realizadas....10
5.1. Prueba de la clase Enlace....11
5.2. Prueba de la clase Enlace Señuelo ....11
5.3. Prueba de la clase Mensaje ....11
5.4. Prueba de la clase Mensaje Controlado ....11
5.5. Prueba de la clase Red Social ....11
5.6. Prueba de la clase Usuario ....12
5.7. Prueba de la clase Usuario Interesado ....12

# 1. Decisiones en el desarrollo

## 1.1. Decisiones de la clase Mensaje

Esta clase posee un String donde almacenar su texto, dos int para sus alcances inicial y actual, y dos Usuarios: su autor y el usuario donde se encuentra. Este último parámetro y su alcance actual son variables a través de la difusión del mensaje, que se hace posible mediante las funciones `cambiarAlcanceDisponible(int nuevoAlcanceDisponible)` y `cambiarUsuarioActual(Usuario nuevoUsuarioActual)`. Para evitar valores `null` en los getters de la clase se incluyen checks a `NullPointerException` en el constructor y en las funciones de cambio.

De cara a la difusión de un mensaje, esta es realizada mediante los métodos de `difusión difunde(Enlace e)` y `difunde(Usuario... usuarios)`, y dos métodos auxiliares `puedeDifundirPor(Enlace e)` y `aceptadoPor(Usuario u)`. La difusión se realiza a traves del segundo `difunde`, que se encarga de pasar el mensaje por la red de tal manera que vaya saltando de usuario a usuario en el orden especificado, siempre que sea posible.

Para ello, buscará el enlace entre ambos usuarios y llamará al otro `difunde` que verá, ayudándose de las funciones auxiliares, si la difusión se puede llevar a cabo, caso en el que la llevará a cabo y devolverá `true` (`false` si no). El método principal recibirá dicho retorno e imprimirá el estado actual de mensaje si la difusión pudo llevarse a cabo.

Por último, especificar que `puedeDifundirPor(Enlace e)` comprueba si el mensaje tiene el alcance necesario para pasar por el enlace mientras que `aceptadoPor(Usuario u)` comprueba si el usuario aceptó el mensaje.

## 1.2. Decisiones en la clase Enlace

Está definida a través de un int con el coste de cada Enlace y otro int estático que almacena el coste de todos los enlaces creados hasta el momento, así como sus Usuarios origen y destino.

Al igual que la clase anterior, evita valores `null` en sus campos a través del try-catch de `NullPointerException`, además, tal y como se especifica en el enunciado, si se quisiera asignar un coste negativo se le asignaría un coste de valor 1.

Posee dos métodos especiales `costeEspecial()` y `costeReal()` que obtienen el valor del coste especial (que será 0 excepto en los enlaces especiales) y el coste real, definido como la suma del coste real y el especial, del enlace.

## 1.3. Decisiones sobre la clase Red Social

Para almacenar los usuarios, enlaces y mensajes, esta clase posee un Hashmap de usuarios cuya clave son sus nombres, un Hashmap de los usuarios de la cadena de difusión asociada a cada mensaje (cuya clave es el propio mensaje) y un ArrayList de mensajes. Se puede acceder a los enlaces a través de los usuarios.

El uso de hashmaps para los usuarios facilita la implementación de las funciones de la fachada ya que reciben como argumentos los nombres de los usuarios a añadir. De cara a la fachada, un usuario puede añadirse, si no preexistía en la red, a través de su nombre y su capacidad de amplificación o sólo mediante su nombre, caso en el que se le asignará una capacidad de valor 2. Los enlaces se añaden mediante los nombres del origen y el destino y su coste. Los mensajes se añaden mediante su texto, alcance disponible y el nombre del autor. Además, pueden añadirse usuarios a la cadena de difusión de un mensaje mediante los paráámetros de creación del mensaje junto con los nombres de los usuarios de la cadena de difusion, que se añadirán al final de dicha cadena.

Por último, el constructor de la red lee los archivos proporcionados, crea la red y difunde el mensaje especificado. Puede guardar su estado inicial mediante la llamada a `saveRedSocialInicial()`.

## 1.4. Decisiones en la clase Usuario

En la clase usuario a parte de las funciones y las variables que se nos pide crear hemos añadido una lista de mensajes, la exposicion y una variable llamada sumaAlcance que guardara el valor de la suma de todos los alcances de menaje.

Para poder trabajar la lista de mensajes desde fuera se han creado varios metodos, como el addMensaje que añade un mensaje a la lista que tiene el usuario, dentro del método se comprueba que el mensaje no se encuentra en la lista para no meter 2 veces el mismo mensaje en la lista, tras eso baja o sube la exposición del usuario dependiendo si es mayor o menor del promedio de alcances (para calcular esto se usa la variable sumaAlcance). Para ello hemos creado la enumeración Exposición que va desde oculta a viral como se pide en el enunciado, para la subida y bajada de exposición se han creado las funciones subir y bajar que cada una hace lo que su nombre indica.

## 1.5 Decisiones clase Usuario Interesado

En el enunciado se pide que el usuario interesado sea como un usuario normal solo que cuando se le pide que se le mande un enlace en vez de devolver lo que se le pide devuelva, si lo tiene, un enlace a un usuario de exposición Alta o Viral, por lo que está en una situación de herencia con el usuario normal y se sobrescriben la funciones getEnlace y primero busca un enlace a un usuario de exposición alta o más y si no lo encuentra hace el proceso normal de getEnlace. Para ello, se hará un ciclo que vaya comprobando todos los usuarios destino de los enlaces y si encuentra alguno con las características pedidas lo devolvera y si no llamara al método padre.

## 1.6 Decisiones clase Enlace Señuelo

Enlace Señuelo se encontrará en una situación de herencia con enlace, añadiendo 2 campos nuevos el factor de coste extra y la probabilidad de retorno obligado. Como en el enunciado se pide que se calcule el coste especial de manera diferente y que el enlace señuelo de vez en cuando devuelva el usuario destino mal se sobrescribirán los métodos *costeEspecial* y *getUsuarioDestino.*

En el método *costeEspecial* en vez devolver 0 como en la clase Enlace devolverá la multiplicación de coste por el factor de coste extra. *GetUsuarioDestino* en vez de devolver siempre el usuario destino tendrá una probabilidad (la variable de probabilidad de retorno obligado) de devolver el usuario origen en su lugar.

## 1.7 Decisiones clase Mensaje Controlado

Mensaje Controlado estará en una situación de herencia con la clase mensaje, añadiendo 1 la variable rigidez a los que ya le vienen heredados de Mensaje. En el enunciado se pide que los mensajes controlados no se puedan difundir por enlaces señuelo y que según la rigidez del mensaje haya un límite a como de expuestos pueden estar los usuarios para que se les mande el mensaje, por ejemplo, para un usuario viral la rigidez tendría que ser de 50 o más, para hacer esto se sobrescribirán los métodos *puedeDifundir* y *aceptadoPor.*

*PuedeDifundir* a parte de lo que suele hacer en la clase padre también comprobara que el enlace recibido no sea un enlace señuelo y en caso de serlo devolverá false. El método *aceptadoPor* comprobara la exposición del usuario recibido y si la rigidez del mensaje cumple para ese usuario, en caso de hacerlo devolverá true, en caso contrario devolverá false.

# 2. Problemas afrontados

## 2.1. Atributos adicionales en la clase Mensaje

Como en el Apartado 5 del enunciado se especifica que el fichero donde está guardado un mensaje sigue la siguiente estructura:

*"Finalmente, el tercer archivo especifica el mensaje que se va a difundir: su autor, el alcance inicial y el usuario en el que se encuentra al comenzar la simulación. Las líneas restantes de este tercer archivo contienen, en el orden dado, los nombres de los usuarios que el mensaje intentará visitar, siguiendo el mecanismo explicado anteriormente en la clase Mensaje."*

El fichero de ejemplo, MENSAJE.txt, posee como cabecera: `"Hola!" 10 ana`. Contrastando ambas informaciones, observamos que el enunciado espera cuatro argumentos en la primera línea del archivo mientras que el fichero posee sólo tres. Así pues, hemos decidido interpretar como que el mensaje siempre se encuentra en su autor al inicio de cada carga de la red social.

Para el posterior guardado de la Red Social, decidimos que se guardara el estado inicial de esta al cargar la red, de tal manera que los archivos guardados contengan la misma información que fue recibida al inicio, incluyendo los nuevos usuarios, enlaces y mensajes.

Así pues, requeríamos que un mensaje almacenase su autor y alcance inicial para poder registrarlo posteriormente, atributs que fueron añadidos directamente a enlace para evitar tener que ocupar memoria extra en Red Social a través de múltiples HashMaps.

## 2.2. Impresión por pantalla al difundir mensajes

En el enunciado se pedía: *"Cada vez que el mensaje consiga propagarse a través de un enlace, el programa deberá mostrar por consola el estado actual del mensaje"*

Por lo que optamos por que el método encargado de difundir un mensaje (`difunde(Usuario... usuarios)`) fuera la encargada también de mostrar dicho estado, si se deseara que esta impresión no se llevara a cabo valdría con añadir un booleano a los parámetros de entrada del método e incluir la impresión dentro de un condicional.

## 2.3. Lectura y escritura de Red Social

Ampliando la cuestión tratada en el apartado anterior, como sólo se carga un mensaje por simulación, la función de guardado registra cada mensaje en un fichero distinto. Similarmente, de cara al código de construcción de la red social, pese a que los mensajes se guardan en un ArrayList para permitir que se añadan varios, en el constructor de la clase sólo se difunde el primero puesto que es el que fue introducido en la carga.

Por comodidad, los ficheros de guardado siempre poseen los nombres “usuarios_save.txt”, “enlaces_save.txt” y “mensaje<nº>_save.txt”, siendo <nº> la posición de dicho mensaje en el ArrayList.

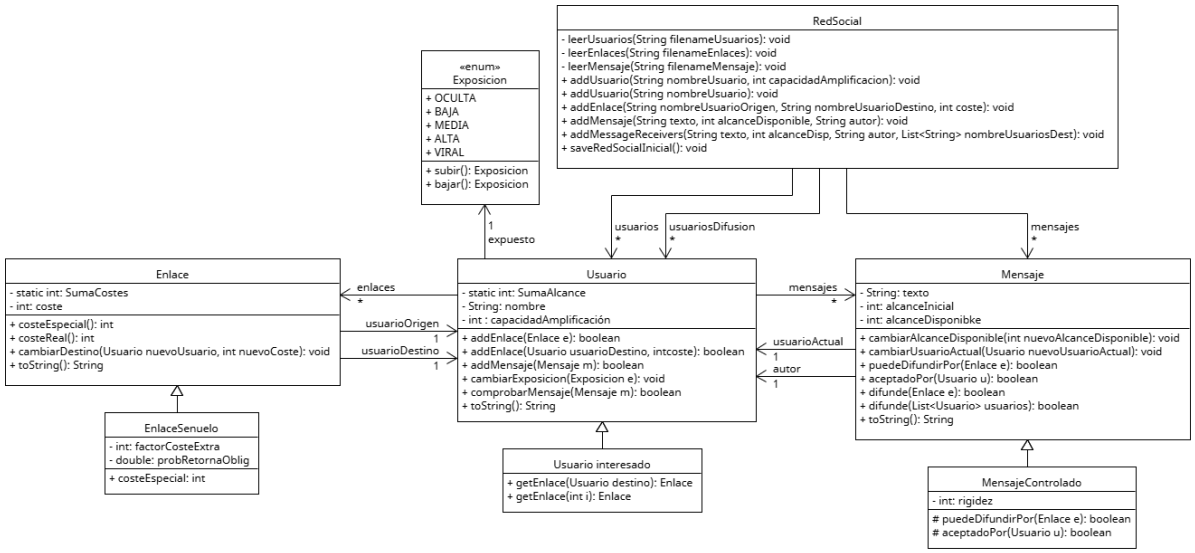
# 3. Problemas pendientes

Si en futuro se quisiera hacer más personalizable, podrían añadirse parámetros extra a los métodos para especificar si se desea:

1. Imprimir el estado actual del mensaje o no
2. Guardar la información de la Red Social en ficheros específicos
3. Difundir los mensajes añadidos

# 4. Diagrama de clases

El diagrama de clases, que viene adjunto en pdf es el siguiente:



# 5. Pruebas realizadas

## 5.1. Prueba de la clase Enlace

Encontrada en el fichero “EjemploDeUsoEnlace.java”, esta prueba consta de la creación de enlaces con valores de coste positivos, negativos y `null` para ver si los dos primeros se guardan con los valores apropiados y si el tercero detecta la entrada inválida.

## 5.2. Prueba de la clase Enlace Señuelo

Encontrada en el fichero “EjemploDeUsoEnlaceSenuelo.java”, esta prueba consta de la creación de un enlace señuelo y llamando diversas veces al método encargado de obtener el usuario destino para ver si cambia pseudoaleatoriamente.

## 5.3. Prueba de la clase Mensaje

Encontrada en el fichero “EjemploDeUsoMensaje.java”, esta prueba basada en la incluida en el enunciado crea enlaces a través de los diversos métodos posibles y difunde un mensaje a través de ellos.

## 5.4. Prueba de la clase Mensaje Controlado

Encontrada en el fichero “EjemploDeUsoMensajeControlado.java”, esta prueba consta de la creación de mensajes controlados para probar cómo cambian las funciones auxiliares de mensaje `puedeDifundirPor` y `aceptadoPor`.

## 5.5. Prueba de la clase Red Social

Encontrada en el fichero “EjemploDeUsoRedSocial.java”, esta prueba se basa en el código proporcionado por el enunciado, incluyendo una llamada a la función de guardado de la Red Social para confirmar que los ficheros de entrada y salida coinciden.

## 5.6. Prueba de la clase Usuario

Encontrada en el fichero “EjemploDeUsoUsuario.java”, esta prueba consta de la creación de usuarios y la modificación de sus atributos para comprobar si cambian adecuadamente mediante la impresión de dichos usuarios.

## 5.7. Prueba de la clase Usuario Interesado

Encontrada en el fichero “EjemploDeUsoUsuarioInteresado.java”, esta prueba funciona similarmente a la de un Usuario.